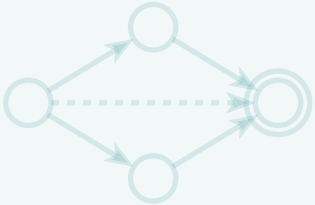
6

Лекция 16–17 марта

Недетерминированные автоматы

Константин Чухарев

Дискретная математика — Весна 2027



Недетерминизм

> *Две дороги разошлись в лесу, и я — я выбрал ту, что меньше была пройдена.*
>
> ***— Роберт Фрост***

Сколько состояний нужно?

Задача: язык слов, у которых третий с конца символ равен 1.

Автомат должен помнить последние три символа.

Пример

Возможных хвостов длины 3 — восемь: $000, 001, \ldots, 111$.

Разным хвостам нужны разные состояния: продолжение 1 разводит слова 00 и 10 — 001 отвергнуто, 101 принято.

Для хвостов длины k состояний нужно 2^k.

Детерминизм имеет цену: память последних k символов — 2^k состояний.

Недетерминизм даёт язык описания, где такая память не строится вручную.

Недетерминированный конечный автомат

Вся разница с ДКА — в функции переходов: она возвращает подмножество состояний, а слагаемое ε разрешает спонтанный шаг без чтения символа.

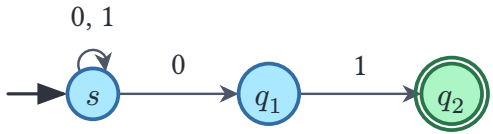
Определение 1 (НКА)

Пятёрка $M = (Q, \Sigma, \delta, q_0, F)$, где

- Q — конечное множество состояний
- Σ — конечный входной алфавит
- $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$ — функция переходов
- $q_0 \in Q$ — начальное состояние
- $F \subseteq Q$ — принимающие состояния

НКА принимает слово, если существует хотя бы один путь в принимающее состояние, остальные пути игнорируются.

НКА для слов, заканчивающихся на «01»



Пример

Одна мысль: на каждом нуле автомат может «подозреть» начало финальной пары и проверить её следующим символом.

Прогон 10101: петля до четвёртого символа, затем $s \to q_1 \to q_2$ — принято.

Прогон 10110: последняя догадка умирает на 0, остальные заканчивают не там — отвергнуто.

Ранняя догадка умирает в q_2 без переходов — для принятия достаточно одной удачной.

Прогон НКА по шагам

Пример

Переходы автомата с рисунка: $\delta(s, 0) = \{s, q_1\}$, $\delta(s, 1) = \{s\}$, $\delta(q_1, 1) = \{q_2\}$.

Прочитано	Множество состояний
ε	$\{s\}$
1	$\{s\}$
0	$\{s, q_1\}$
0	$\{s, q_1\}$
1	$\{s, q_2\}$

Пример

Финальное множество для 1001 содержит $q_2 \in F$ — принято, а траектория слова 1000 кончается в $\{s, q_1\}$ — отвергнуто.

Что значит «угадывает»?

Недетерминизм часто описывают словом «угадывание».

Это опасная метафора.

В определении языка действует только квантор существования: слово принято, если *существует* успешный путь.

Никакой случайности, никакого параллелизма, никакого алгоритма перебора: НКА — это декларативное описание языка.

Исполнительная сторона: все пути за один раз прогоняют симулятор множеств и конструкция подмножеств.

Сколько путей у НКА?

Каждая догадка — ветвь вычисления.

Число путей растёт с длиной слова стремительно: на каждом ветвлении пути удваиваются.

Пример (Две параллельные ветви)

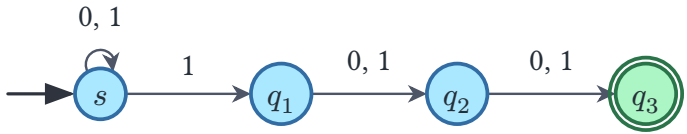
Старт s, по 0 ведут два перехода — в p и в q. Из p и из q каждый символ ведёт снова в оба состояния, а сами p и q — принимающие.

Слово 0^n даёт ровно 2^n путей — на каждом шаге путь волен выбрать p или q. Язык при этом простой — слова, начинающиеся с нуля.

Разных множеств состояний НКА при этом всего 2^k, где $k = |Q|$.

Все пути сразу умещаются в конечный объект: множество состояний, где НКА может находиться после чтения префикса.

Третий с конца символ равен 1



Пример

Догадка начинается на единице и отсчитывает три символа до конца.

Прогон 1101: $\{s\} \to \{s, q_1\} \to \{s, q_1, q_2\} \to \{s, q_2, q_3\} \to \{s, q_1, q_3\}$ — принято.

Прогон 1001: $\{s\} \to \{s, q_1\} \to \{s, q_2\} \to \{s, q_3\} \to \{s, q_1\}$ — отвергнуто.

НКА обходится четырьмя состояниями при любой длине слова, а минимальному ДКА нужно $2^3 = 8$: он обязан различать все хвосты длины 3.

Почему именно 8 — ответит теорема Майхилла–Нерода.

Эпсилон-переходы

ε-переход — смена состояния без чтения символа, бесплатный шаг вычисления.

Из	по a	по b	по ε
q_0	—	—	q_1
q_1	q_1	—	q_2
q_2	—	q_2	—

Пример (Автомат для языка a^*b^*)

Состояния q_0, q_1, q_2: старт q_0, приём q_2, ε-рёбра из q_0 в q_1 и из q_1 в q_2.

Зона q_1 читает блок a, зона q_2 — блок b, и автомат принимает ровно a^*b^*: вернуться из q_2 в q_1 нельзя.

Эпсилон-замыкание

ε-рёбра бесплатны, и живое множество нужно замыкать.

Определение 2 (ε-замыкание)

ε-замыкание множества состояний S, обозначается ε -closure(S), — множество всех состояний, достижимых из S по цепочкам ε-переходов, включая пустую: сами состояния S лежат в замыкании.

Пример (Замыкание обходом с очередью)

1. q_0: ребро $q_0 \to q_1$ — множество $\{q_0, q_1\}$, очередь q_1.
2. q_1: ребро $q_1 \to q_2$ — множество $\{q_0, q_1, q_2\}$, очередь q_2.
3. q_2: ε-рёбер нет — очередь пуста, замыкание собрано.

Прогон слова aab: $\{q_0, q_1, q_2\} \to \{q_1, q_2\} \to \{q_1, q_2\} \to \{q_2\}$ — принято, конец в принимающем q_2.

Шаг чтения с эпсилон-рёбрами

Пример

Прогон слова ba тем же способом: $\{q_0, q_1, q_2\} \to \{q_2\} \to \varnothing$.

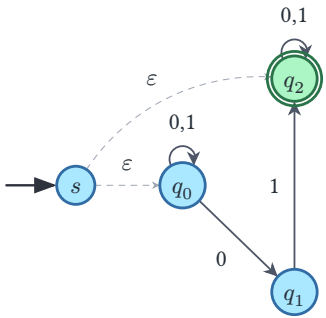
После буквы a живых путей не осталось — слово отвергнуто, дочитывать его незачем.

Шаг чтения для автомата с ε-рёбрами — всегда два действия: собрать преемников по символу из всех состояний множества, затем замкнуть результат по ε:

$$\hat{\delta}(S, a) = \varepsilon\text{-closure}\left(\bigcup_{q \in S} \delta(q, a)\right)$$

Стартовое множество тоже замыкается: работа начинается не с $\{q_0\}$, а с ε-closure$(\{q_0\})$.

Автомат с эпсилон-ребром



Пример (Прогон)

Прогон 01: $s \to q_0 \to q_1 \to q_2$ по ε, 0, 1 — принято. Пустое слово тоже принято: из s в q_2 ведёт цепочка ε-рёбер.

Ловушка: неосторожный эпсилон-переход

Пример

Автомат собран из ветви поиска через q_0 и q_1, петель дочитывания в q_2 и ε-ребра в q_2, заведённого ради пустого слова.

Пустое слово действительно принимается: из s в q_2 можно попасть, не прочитав ни символа.

Язык вырос до Σ^*

Ребро по ε из s в принимающее q_2 с петлями разрешает дочитать после него что угодно: пустое слово, пару 01, любой мусор — принимающий путь есть у каждого слова.

Правильная склейка: пустое слово обслуживает отдельное принимающее состояние без исходящих рёбер.

Конструкция подмножеств

> *Бог не играет в кости.*
>
> ***— Альберт Эйнштейн***

Эквивалентность ДКА и НКА

Теорема 1 (Рабин — Скотт, 1959)

Для любого НКА существует ДКА, распознающий тот же язык.

Доказательство

Докажем построением: по НКА N соберём ДКА с состояниями-подмножествами Q. Начальное состояние — ε -closure$(\{q_0\})$: НКА уже прошёл все ε-рёбра от старта. Переход собирает всё, что может случиться за символ:

$$\delta'(S, a) = \varepsilon \text{ -closure} \left(\bigcup_{q \in S} \delta(q, a) \right).$$

Принимающие состояния — множества, пересекающие F, а замыкание в обеих формулах обязательно: ε-рёбра бесплатны.

Корректность конструкции подмножеств

Конструкция определена, осталось убедиться, что она принимает ровно слова НКА.

Доказательство

Докажем индукцией по длине слова: инвариант — после чтения слова w ДКА стоит в множестве всех состояний НКА, достижимых из q_0 по w с ε-довесками.

База. До чтения ДКА стоит ровно в ε -closure$(\{q_0\})$ — все состояния, настигнутые ε-рёбрами от старта. **Шаг.** Переход $\delta'(S, a)$ собирает преемников всех состояний S и замыкает их по ε, поэтому инвариант переносится с w на wa.

Слово принимается ДКА ровно тогда, когда финальное множество пересекает F, то есть когда у НКА есть успешный путь.

Тот же инвариант работает в симуляторе: множества на лету — это состояния ДКА, которые можно не строить заранее.

Рецепт детерминизации

Теорема обещает ДКА, рецепт собирает его руками. Множества открываются по одному, как вершины при обходе. Каждое получит строку таблицы переходов.

1. Начать с ε-замыкания начального множества $\{q_0\}$.
2. Для каждого построенного множества S и символа a собрать все переходы по a из состояний S и замкнуть результат по ε.
3. Если такое множество уже встречалось — провести дугу в него, иначе завести новое состояние.
4. Закончить, когда у всех состояний заполнены переходы.
5. Принимающими объявить множества, содержащие хотя бы одно принимающее состояние НКА.

Рецепт всегда завершается: новых множеств не больше $2^{|Q|}$.

Детерминизация на примере

Применим рецепт к НКА для языка «заканчивается на 01» — множества открываются по ходу обхода.

Пример (Порядок открытия)

Ведём очередь множеств и заполняем переходы, пока появляются новые.

1. Старт: $S_0 = \{s\}$. По 0 из s расходятся два перехода: $\{s, q_1\}$ — новое (S_1). По 1 петля даёт $\{s\}$ — уже открыто.
2. $S_1 = \{s, q_1\}$. По 0: из q_1 переходов нет, снова $\{s, q_1\}$. По 1: ребро в q_2 даёт $\{s, q_2\}$ — новое (S_2).
3. $S_2 = \{s, q_2\}$. По 0: $\{s, q_1\}$, по 1: $\{s\}$ — оба старые, очередь пуста, ДКА готов.

Из восьми подмножеств достижимы три, пустого среди них нет.

Таблица переходов ДКА

Строки — в порядке открытия множеств.

Множество S	$\delta'(S, 0)$	$\delta'(S, 1)$
$\{s\}$	$\{s, q_1\}$	$\{s\}$
$\{s, q_1\}$	$\{s, q_1\}$	$\{s, q_2\}$
$\{s, q_2\}$	$\{s, q_1\}$	$\{s\}$

Пример

Прогон 10110: $\{s\} \to \{s\} \to \{s, q_1\} \to \{s, q_2\} \to \{s\} \to \{s, q_1\}$ — отвергнуто.

Принимающее одно — $\{s, q_2\}$: только оно содержит q_2 из F.

Детерминизация эпсилон-автомата

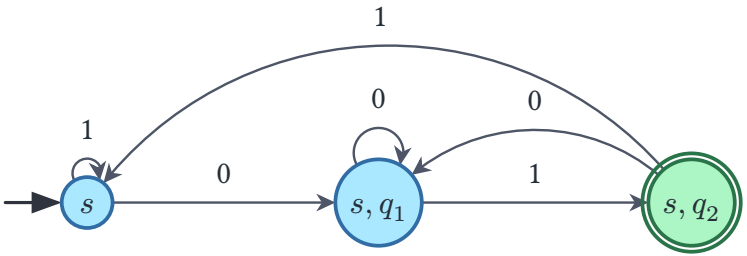
Множество S	$\delta'(S, a)$	$\delta'(S, b)$
$\{q_0, q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$\{q_1, q_2\}$	$\{q_1, q_2\}$	$\{q_2\}$
$\{q_2\}$	$\emptyset$	$\{q_2\}$
$\emptyset$	$\emptyset$	$\emptyset$

Пример

Первая строка по шагам: по a из $\{q_0, q_1, q_2\}$ доступна только петля q_1, и замыкание добавляет q_2. Прогон ba: $\{q_0, q_1, q_2\} \to \{q_2\} \to \emptyset$ — отвергнуто на пустом множестве.

Непустые состояния — принимающие, потому что язык a^*b^* замкнут относительно взятия префикса.

Множества-состояния ДКА «01»



Пример

Множества-состояния для НКА, ищущего пару 01: $\{s\}$ — ожидание, $\{s, q_1\}$ — «кончается на 0», $\{s, q_2\}$ — «кончается на 01».

Это в точности ДКА для языка «заканчивается на 01»: имена-множества играют те же роли, что q_0, q_1, q_2.

Симулятор НКА: замыкание

Обход в глубину

Замыкание — обход в глубину по ε-рёбрам:

```
def eps_closure(states, trans):
    seen, stack = set(states), list(states)
    while stack:
        q = stack.pop()
        for r in trans.get((q, ""), []):
            if r not in seen:
                seen.add(r)
                stack.append(r)
    return frozenset(seen)
```

Множество при замыкании только растёт, поэтому цикл завершается за линейное время.

Симулятор НКА: шаг чтения

Шаг чтения собирает преемников по символу и сразу замыкает их:

```
def run(trans, start, accept, word):
    S = eps_closure({start}, trans)
    print("start:", sorted(S))
    for a in word:
        nxt = set()
        for q in S:
            nxt.update(trans.get((q, a), []))
        S = eps_closure(nxt, trans)
        print(f"{a} ->", sorted(S))
    return bool(S & accept)
```

Таблица ДКА здесь не строится вовсе: множества считаются на лету, ровно для одного слова.

Прогон на «1001»

Пример

```
ends01 = {("s", "0"): ["s", "q1"],
          ("s", "1"): ["s"],
          ("q1", "1"): ["q2"]}
run(ends01, "s", {"q2"}, "1001")
```

```
start: ['s']
1 -> ['s']
0 -> ['q1', 's']
0 -> ['q1', 's']
1 -> ['q2', 's']
```

Ответ `True`: финальное множество содержит q_2.

Восемь состояний из шестнадцати

Детерминизация автомата «третий с конца» порождает всю таблицу:

Множество S	$\delta'(S,0)$	$\delta'(S,1)$
$\{s\}$	$\{s\}$	$\{s,q_1\}$
$\{s,q_1\}$	$\{s,q_2\}$	$\{s,q_1,q_2\}$
$\{s,q_2\}$	$\{s,q_3\}$	$\{s,q_1,q_3\}$
$\{s,q_1,q_2\}$	$\{s,q_2,q_3\}$	$\{s,q_1,q_2,q_3\}$
$\{s,q_3\}$	$\{s\}$	$\{s,q_1\}$
$\{s,q_1,q_3\}$	$\{s,q_2\}$	$\{s,q_1,q_2,q_3\}$
$\{s,q_2,q_3\}$	$\{s,q_3\}$	$\{s,q_1,q_3\}$
$\{s,q_1,q_2,q_3\}$	$\{s,q_2,q_3\}$	$\{s,q_1,q_2,q_3\}$

Строки в порядке открытия, принимающие — нижние четыре: они содержат q_3.

Экспоненциальная цена детерминизма

Пример

Язык «k-й с конца символ равен 1»: НКА обходится $k+1$ состояниями, а минимальный ДКА требует 2^k.

При $k = 10$ — одиннадцать состояний против 1024, при $k = 30$ — тридцать одно против $2^{30} \approx 10^9$.

Булеан конечен, поэтому детерминизация всегда заканчивается. Цена — экспонента, арифметика булеана.

Недетерминизм в конечных автоматах устраняем, а для моделей с неограниченной памятью вопрос «можно ли убрать его без экспоненты» — центральная загадка теории сложности.

ДКА и НКА: сравнение

Свойство	ДКА	НКА
Переход	одно состояние	множество состояний
ε-переходы	запрещены	разрешены
Пути вычисления	ровно один	хотя бы один успешный
Дополнение языка	сменить F	сначала детерминизация
Компактность	может расти как 2^k	обычно линейная

Разница между моделями — в цене описания, а не в классе языков.

Каждый НКА превращается в ДКА, и обратное тривиально: ДКА — частный случай НКА.

Ловушка: дополнение НКА

Инверсия приёма

У ДКА дополнение языка — поменять принимающие состояния местами с отвергающими.

У НКА приём не работает: принятие — существование принимающего пути, а дополнение требует отсутствия *всех* принимающих путей.

Пример

Слово, у которого один путь успешен и десять неуспешных, НКА принимает, а автомат с «инвертированной» F отвергнет.

Правильный путь: сначала детерминизация, затем смена F — это единственная операция лекции, которой нужен ДКА.

Недетерминизм в работе

> *Когда сомневаешься — применяй грубую силу.*
>
> ***— Кен Томпсон***

Склейки из эпсилон-переходов

Регулярное выражение собирается из автоматов-кирпичей, и все три склейки делаются ε-рёбрами.

Выражение	Склейка
$r \mid s$	новые старт и конец, ε-рёбра к обоим кирпичам и от их концов
rs	ε-ребро из конца кирпича r в старт кирпича s
r^*	ε-петля возврата в старт кирпича и ε-обход мимо цикла

Базы тоже просты: символ a — два состояния и ребро по a, слово ε — два состояния и ε-ребро, язык $\emptyset$ — два состояния без рёбер.

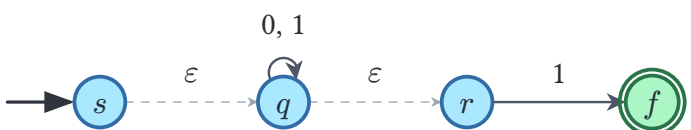
Кирпичи при склейках не перестраиваются, поэтому сборка идёт рекурсивно по дереву выражения.

Сборка автомата для $(0 \mid 1)^*1$

Пример

Соберём автомат по таблице склеек:

1. База 1: два состояния и ребро по 1.
2. Класс $0 \mid 1$: состояние q с петлями по 0 и 1.
3. Звезда $(0 \mid 1)^*$: ε-ребро в цикл и ε-ребро из него.
4. Конкатенация: ε-ребро из цикла в базу 1, её конец принимающий.



Звезда проверяется сразу: пустое слово проходит по ε-рёбрам из s через q в r, но r не принимающее — для приёма нужна ещё буква 1.

Прогон автомата Томпсона

Пример

Слово 001 по автомату с рисунка.

Старт: ε -closure$(\{s\}) = \{s, q, r\}$ — цикл и его ε-выход видны ещё до первого символа.

Дальше два нуля подряд: живо одно и то же $\{q, r\}$.

После 1: из q петля ведёт в q, из r ребро ведёт в f, а замыкание возвращает r по ε-ребру из q — живо множество $\{q, f, r\}$, и слово кончилось в принимающем состоянии.

Состояние r живёт во множестве с самого старта: автомат параллельно ждёт цикл и последний символ 1.

Слово 010 отвергается: последний 0 стирает приёмник f из живого множества, и остаются только $\{q, r\}$ — ни одного принимающего.

Замкнутость: три склейки

Теорема 2 (Замкнутость относительно объединения, конкатенации и звезды)

Если L_1 и L_2 распознаются НКА, то НКА распознают и $L_1 \cup L_2$, и $L_1 L_2$, и L_1^*.

Доказательство

Докажем построением: каждая операция — новый ε-НКА из копий автоматов.

Объединение. Старт s с ε-рёбрами в оба старта копий, концы копий — в новый принимающий: путь читает слово одной копией.

Конкатенация. Приём — конец второй копии, а конец первой соединён ε-ребром со стартом второй: принимающий путь читает слово вида uv, где $u \in L_1$, $v \in L_2$.

Звезда. Свежие s, f с ε-рёбрами $s \to s_1$, $f_1 \to s_1$, $f_1 \to f$, $s \to f$: петля крутит цикл, обход даёт ε.

Выражение в конвейере лексера

Пример (Конвейер лексера)

Каждый токен компилятора — регулярное выражение: идентификатор, число, строковый литерал.

Конструкция Томпсона собирает выражения в НКА, конструкция подмножеств детерминизирует их в ДКА, и дальше каждый символ — один табличный переход за $O(1)$.

Недетерминизм — язык описания, детерминизм — форма исполнения.

Движки с возвратами (backtracking) на злых выражениях взрываются по времени, поэтому линейные движки вроде `regex` в Rust строят автомат, а не перебирают пути.

Поиск подстроки как угадывание

Пример (НКА для языка «содержит 101»)

Петли в s откладывают догадку, цепочка $s \to q_1 \to q_2 \to q_3$ проверяет шаблон, петли в q_3 дочитывают хвост.

Ключевой переход $\delta(s, 1) = \{s, q_1\}$ позволяет продолжить поиск или начать проверку.

Прогон слова 11011:

$$\{s\} \to \{s, q_1\} \to \{s, q_1\} \to \{s, q_2\}$$
$$\to \{s, q_1, q_3\} \to \{s, q_1, q_3\}$$

В финальном множестве есть q_3 — принято.

Симуляция множеств читает текст одним проходом без возвратов — недетерминированный поиск, чья битовая упаковка известна как алгоритм Shift-And.

Когда счётчик лучше угадывания?

Пример (Двоичная запись делится на 3)

О префиксе достаточно помнить остаток по модулю 3: дописывание бита b превращает остаток r в $(2r + b) \bmod 3$.

Состояния — остатки r_0, r_1, r_2: старт и принимающее — r_0, переход по биту b из r_i ведёт в $r_{(2i+b) \bmod 3}$.

Прогон 110: $r_0 \to r_1 \to r_0 \to r_0$ — принято, и верно: $6 = 2 \cdot 3$.

Прогон 1011 кончается в r_2: одиннадцать на три не делится.

Угадывать здесь нечего: свойство всего слова — один арифметический инвариант.

Недетерминизм окупается там, где свойство звучит как «где-то», «хотя бы одна», «одна из альтернатив».

Что мы поняли?

НКА распознают тот же класс языков, что и ДКА, — разница в цене описания.

- НКА: переход возвращает множество состояний, принять — существует успешный путь.
- ε-переходы бесплатны, и ε-замыкание нужно учитывать на каждом шаге чтения.
- Конструкция подмножеств симулирует все пути сразу: состояния ДКА — множества состояний НКА.
- Экспонента 2^k бывает настоящей: язык «третий с конца» требует всех восьми множеств из шестнадцати.
- Томпсон собирает автомат для выражения ε-склейками, и те же склейки доказывают замкнутость класса.
- Дополнение — единственная операция, требующая детерминизации: недетерминизм устраняем, но не бесплатно.

Вопросы для самопроверки

Вопросы для самопроверки — недетерминизм и детерминизация

1. Почему «угадывает» — плохое описание НКА? Дайте формулировку через квантор.
2. Язык «третий с конца равен 1»: сколько состояний у НКА и у минимального ДКА?
3. Какие два действия составляют шаг чтения с ε-переходами, и что сломается, если в конструкции подмножеств забыть замыкание?
4. Почему в детерминизации автомата «третий с конца» достижимы ровно восемь множеств из шестнадцати?
5. Перечислите склейки конструкции Томпсона. Где в каждой работает ε-ребро?
6. Почему для дополнения языка НКА мало поменять принимающие состояния?

Что почитать

- m23 — «Недетерминированные конечные автоматы», «Конструкция подмножеств», «Регулярные выражения».